

FASE 02 – CRUD SEQUELIZE

Nessa fase, criaremos programas javascript para ENTENDERMOS* os métodos SEQUELIZE equivalentes aos comandos SQL para realizarmos as atividades CRUD:

*** as construções desse exemplo são apenas para fins didáticos (aprendizagem).**

Posteriormente, na próxima fase, serão reestruturadas com parâmetros adequados e colocados em métodos CRUD HTTP

1) CREATE

SQL:

```
INSERT INTO Bookmarks(id, site, detalhes, link, createdat, updatedat)
VALUES (21, 'OpenAI', 'Plataforma de IA com várias ferramentas e APIs', 'https://openai.com',
DATETIME('now'), DATETIME('now'))
```

SEQUELIZE:

Crie o programa `Sequelize-Insert.js` para realizar o equivalente ao comando SQL de Inserção:

```
-----
const Bookmark = require('./models').Bookmark;
Bookmark.create({
  id: 21,
  site: 'OpenAI',
  detalhes: 'Plataforma de IA com várias ferramentas e APIs',
  link: 'https://openai.com' })
.then(bookmark => {
  console.log("Bookmark inserido com sucesso:", bookmark.toJSON()); })
.catch(error => { console.error("Erro ao inserir o bookmark:", error); });
-----
```

Salve e execute com o comando `node Sequelize-Insert.js`

Você pode repetir a execução mudando os dados (lembre-se que o campo id deve ser único)

2) READ - duas formas de consultas:

2.1) Consultar múltiplos registros (10) a partir de um deslocamento de início (0)

SQL:

```
SELECT id, site, detalhes, link, createdat, updatedat FROM Bookmarks LIMIT 10 OFFSET 0;
```

SEQUELIZE:

Crie o programa **Sequelize-SelectAll.js** para realizar o equivalente ao comando SQL de consulta:

```
-----  
  
const { Bookmark } = require('./models'); // Importa o modelo Bookmark  
  
async function selectAll(limit = 10, offset = 0) {  
  try {  
    const bookmarks = await Bookmark.findAll({  
      limit: limit,    // Limita a quantidade de registros  
      offset: offset  // Desloca a partir de quantos registros devem ser pulados  
    });  
    // Converte cada registro para um objeto simples  
    const bookmarkData = bookmarks.map(bookmark => bookmark.toJSON());  
    console.log(bookmarkData);  
  } catch (error) {  
    console.error("Erro ao buscar bookmarks:", error);  
  }  
}
```

```
selectAll() // consulta com valores default limit = 10, offset = 0
```

```
selectAll(5,15) // consulta com valores default limit = 5, offset = 15
```

Salve e execute com o comando **node Sequelize-SelectAll.js**

Você pode repetir a execução mudando os parâmetros

2.2) Consultar registro passando chave - parâmetro id

SQL:

```
SELECT id, site, detalhes, link, createdat, updatedat FROM Bookmarks WHERE id=1;
```

SEQUELIZE:

Crie o programa `Sequelize-SelectID.js` para realizar o equivalente ao comando SQL de consulta:

```
-----  
const { Bookmark } = require('./models'); // Importa o modelo Bookmark
```

```
async function selectOne(id) {  
  try {  
    const bookmark = await Bookmark.findByPk(id);  
  
    if (bookmark) console.log(bookmark.toJSON());  
    else console.log(`Chave: ${id} inexistente`);  
  
  } catch (error) {  
    console.error("Erro ao buscar bookmark:", error);  
  }  
}
```

```
selectOne(1) // consulta registro id=1
```

```
selectOne(15) // consulta registro id=15
```

```
-----  
Salve e execute com o comando node Sequelize-SelectID.js
```

Você pode repetir a execução mudando os parâmetros

3) UPDATE

SQL:

```
UPDATE Bookmarks SET site='xxxx', detalhes='YYYYYY' WHERE id=1;
```

SEQUELIZE:

Crie o programa `Sequelize-Update.js` para realizar o equivalente ao comando SQL de alteração:

```
-----  
  
const { Bookmark } = require('./models'); // Importa o modelo Bookmark  
  
async function updateId() {  
  try {  
    const updatedData = {  
      // Campos e novos valores a serem atualizados  
      site: 'xxxx',  
      detalhes: 'YYYYYY'  
    };  
  
    const [affectedRows] = await Bookmark.update(  
      updatedData, // Dados a serem atualizados  
      { where: { id: 1 } } // Condição para selecionar o registro  
    );  
  
    if (affectedRows > 0) {  
      console.log('Registro atualizado com sucesso.');    } else {  
      console.log('Nenhum registro encontrado para o ID fornecido.');    }  
  } catch (error) {  
    console.error('Erro ao atualizar o registro:', error);  
  }  
}
```

```
updateId();  
  
-----
```

Salve e execute com o comando `node Sequelize-Update.js`

Você pode repetir a execução mudando os dados (valores e chave)

4) DELETE - uma rota com o Método HTTP DELETE - receberá parâmetro: id - chave do registro e apagará o registro

SQL:

DELETE FROM Bookmarks WHERE id=1;

SEQUELIZE:

Crie o programa **Sequelize-Delete.js** para realizar o equivalente ao comando SQL de exclusão:

```
-----

const { Bookmark } = require('./models'); // Importa o modelo Bookmark

async function deleteId(id) {
  try {
    const affectedRows = await Bookmark.destroy({
      where: { id: id } // Condição para selecionar o registro
    });

    if (affectedRows > 0) {
      console.log('Registro excluído com sucesso.');
```

Salve e execute com o comando **node Sequelize-Delete.js**

Você pode repetir a execução mudando os dados (valores e chave)

PRÓXIMOS PASSOS:

- 1) Criar as ROTAS CRUD para os métodos HTTP com o EXPRESS mapeando nas rotinas CRUD SEQUELIZE;
- 2) Criar os EJS para as consultas 2.1 e 2.2;

Como PLUS, você poderá implementar com MIDWARE a VALIDAÇÃO DOS CAMPOS (AJV).